

NATONAL ECONOMICS UNIVERSITY

School of Advanced Education Program

Network and Computer Technology

TRESGO: A Web Interview Recorder

Author:

Pham Mai Phuong - ID: 11247218

Vu Kim Minh - ID: 11247205

Nguyen Thi Thuy Linh - ID: 11247188

Supervisor:

Tran Hung



Individual Contributions

Member	Contribution (%)
Pham Mai Phuong - ID: 11247218	33.33%
Vu Kim Minh - ID: 11247205	33.33%
Nguyen Thi Thuy Linh - ID: 11247188	33.33%

Github: <https://github.com/vuminh18/TresGo-Interview-Recorder.git>

Video: <https://url-shortener.me/2UV8>

Contents

1	INTRODUCTION	4
1.1	Context and Problem Statement	4
1.1.1	Context	4
1.1.2	Problem	4
1.1.3	Proposed Solution	4
2	SYSTEM DESIGN	5
2.1	System Architecture	5
2.1.1	Model and Protocols	5
2.1.2	Operational Workflow	5
2.2	Technological Stack	5
2.3	Data Storage Structure	6
2.3.1	Directory Hierarchy	6
2.3.2	File Naming Convention	7
2.3.3	Metadata (<code>meta.json</code>)	7
3	IMPLEMENTATION	8
3.1	Project Workflow (Step-by-Step)	8
3.2	Speech-to-Text Feature (Bonus)	8
3.2.1	Technology and Operation	8
3.2.2	Accuracy Optimization	8
3.2.3	Output	8
4	SECURITY AND RELIABILITY (UX)	10
4.1	Security Measures	10
4.1.1	Server-Side Validation	10
4.1.2	HTTPS/SSL Encryption	10
4.2	Error Handling & Reliability (UX)	11
4.2.1	Manual Retry Policy	11
4.2.2	Input Validation (Client-Side)	11
5	EXPERIMENTAL RESULTS	12
6	SELF-CHECK QUESTIONS	15
6.1	Why must the Token be validated on the server?	15
6.2	What are the benefits of per-question recording and upload?	15
6.3	Why is HTTPS required publicly?	15
6.4	How do you design the retry/backoff mechanism?	16
6.5	Which Speech-to-Text engine and why?	16

7	CONCLUSION AND FUTURE DEVELOPMENT	17
7.1	Conclusion	17
7.2	Limitations	17
7.3	Future Development	17

Chapter 1

INTRODUCTION

1.1 Context and Problem Statement

1.1.1 Context

In the era of global economic integration and the rapid advancement of information technology, the remote working model has become increasingly prevalent. This shift has driven a growing demand for Remote Interviews, especially the asynchronous video interview format, where candidates record video responses at their convenience. This method offers significant benefits by saving time, reducing travel costs, and expanding the global pool of potential candidates.

1.1.2 Problem

However, traditional video interview systems suffer from a major drawback: they typically require continuous recording for the entire session (lasting 15–20 minutes), compiling everything into a single long video file for a final server upload. This approach poses critical data risks and may effect the user experience:

- **High Risk of Data Loss:** If a candidate encounters a technical failure (network disconnect, unstable connection, or power loss of the device) in the final minutes of the interview, all recorded data is at high risk of being completely lost and unrecoverable.
- **Poor Candidate Experience:** The need to re-record the entire process due to technical glitches leads to wasted time, increased candidate stress, and a diminished quality of the overall recruitment experience.

1.1.3 Proposed Solution

To fundamentally address this core vulnerability and enhance the reliability of the remote interview process, the TresGo project proposes a solution based on the Per-Question Upload mechanism. Instead of relying on a single final upload, the system will automatically process and upload the video response immediately after the candidate finishes answering each question. This strategy ensures that data is saved securely as soon as it is created, minimizing the risk of accidental data loss.

Chapter 2

SYSTEM DESIGN

2.1 System Architecture

2.1.1 Model and Protocols

The TresGo system is fundamentally structured around a conventional Client-Server Architecture, which is designed to clearly separate the user interface (Client) from the core business logic and data persistence layers (Server).

The primary data transfer protocols utilized are HTTP/HTTPS. Critically, during the development and testing phases, the project relies on the Ngrok utility to establish a secure, publicly accessible HTTPS tunnel to the local FastAPI server. This secure connection is mandatory, as modern web browsers strictly require HTTPS to activate the MediaStream Recording API for accessing a candidate's camera and microphone hardware.

2.1.2 Operational Workflow

The system's operation is based on an event-driven principle, strictly adhering to the "Per-Question Upload" mechanism. The workflow is executed in three main stages:

- **Client Recording:** The candidate records a response to a specific question.
- **Upload:** Immediately upon completion of the response, the Client by using Fetch API starts the asynchronous transfer of the video file to the FastAPI Server.
- **Server Processing:** The Server secures the file in Disk Storage and concurrently initiates a background task for the AI Engine.

2.2 Technological Stack

The project leverages a robust and efficient technological stack, carefully selected to support the asynchronous and data-intensive nature of the application.

Table 2.1: Technological Stack of the TresGo System

Component	Key Technology	Role and Rationale
Frontend	HTML5, CSS3, Vanilla JavaScript	Develop user interfaces (login, interview). Ensure maximum compatibility and performance.
Recording API	MediaStream Recording API	Enables native access to device camera and microphone, capturing video streams directly within the browser.
Data Transfer	Fetch API	Used to execute asynchronous data transfer of individual video files (Per-Question Upload) to the backend, which prevents UI blocking.
Backend Framework	Python 3.12 & FastAPI	FastAPI is chosen for its high performance, support for asynchronous programming (async/await), and its suitability for heavy I/O tasks such as file uploads. Uvicorn is used as the Server Gateway Interface (ASGI) server.
AI Engine (Bonus)	OpenAI Whisper (Base Model)	Integrated as an offline processing tool to automatically convert the audio track of each video into a text Transcript (Speech-to-Text). This task is performed non-blocking in a background process.

2.3 Data Storage Structure

The system utilizes local disk storage, which is structured hierarchically to ensure logical organization, easy management, and quick retrieval of individual interview sessions.

2.3.1 Directory Hierarchy

All data is stored in the root folder `uploads/`. Each candidate's interview session will be saved in a separate folder according to the following rule:

`uploads/DD_MM_YYYY_HH_nn_Ten_User/`

Example: `uploads/13_12_2025_01_30_Nguyen_Thi_Thuy_Linh/`

2.3.2 File Naming Convention

A standardized file naming convention is crucial for reflecting the sequential order of the answers, which is the cornerstone of the system's architecture:

- **Video Responses:** Files are named sequentially according to the question index.
 - Norm: `Q[Question Index].webm`
 - Example: `Q1.webm`, `Q2.webm`, etc.
- **Transcript Files:** The text output generated by the Whisper AI Engine is stored alongside its corresponding video file.
 - Norm: `Q[Question Index].txt`.
 - Example: `Q1.txt`, `Q2.txt`, etc.

2.3.3 Metadata (`meta.json`)

Every session folder includes a single `meta.json` file. This file stores essential metadata to support system administration and data integrity checks, including:

- **Session Details:**
 - `status`: `started`, `finished`, etc.
 - `startTime`: The start time of the interview.
 - `endTime`: The time when the interview ends.
 - `userInfo`: Candidate name, token
- **Upload Log:** A structured record of every successfully uploaded video file, including the question index and timestamp (`uploadedAt`), confirming the successful implementation of the Per-Question Upload mechanism.

Chapter 3

IMPLEMENTATION

3.1 Project Workflow (Step-by-Step)

The TresGo interview process is designed as a sequence of five clear steps to ensure data integrity and a smooth user experience.

3.2 Speech-to-Text Feature (Bonus)

This is an advanced feature designed to help recruiters by providing a text version (transcript) of every video answer.

3.2.1 Technology and Operation

- **Library:** We use the powerful OpenAI Whisper library with the base model.
- **Operation:** The model runs **offline** on the Server, embedded within a FastAPI **Background Task**. This means the system does not depend on an external third-party API for transcription, ensuring speed and data privacy.

3.2.2 Accuracy Optimization

To achieve higher accuracy for the transcriptions, especially with mixed languages (English and Vietnamese), we use a simple technique called **Prompting**.

- **Method:** Before processing the audio, we feed the Whisper model a specific technical Prompt (e.g., "This is an interview answer for a job application...").
- **Result:** This technique provides the AI with the necessary context, greatly improving its ability to accurately transcribe the professional language and specific terminology used in the interview setting.

3.2.3 Output

After processing, each `Qx.webm` video will generate a corresponding text file `Qx.txt` containing the transcript, stored in the same interview folder.

Table 3.1: TresGo Interview Workflow Steps

Step	Action Taken	Location (Client/Server)	Goal
Step 1: Authentication	The candidate enters their Token and Name. The Client sends this data to the Server using the API <code>/api/verify-token</code> .	Client (JS Fetch API) → Server (FastAPI)	To confirm the Token is valid and grant access to start the interview.
Step 2: Session Setup	The Client calls the API <code>/api/session/start</code> .	Server (FastAPI)	The Server immediately creates a unique folder (e.g., <code>uploads/12_12_2025</code>) on the disk to save all the files for this specific candidate.
Step 3: Interview & Upload	This is the main part, implementing the Per-Question Upload.	Client (JS MediaStream API) → Server	To save data immediately after each answer, preventing data loss.
3a. Recording	The Client displays the question. The candidate records their video answer.	–	–
3b. Uploading	The candidate clicks "Next". The Client stops the recording and immediately sends the video file to the Server via the API <code>/api/upload-one</code> .	–	–
Step 4: Background Processing	The Server receives the file, saves it to the unique folder, and quickly returns a simple "OK" message to the Client.	Server (FastAPI)	To avoid making the user wait. The Server activates the Whisper AI task in the background (non-blocking) to start transcribing the video later.
Step 5: Session Finalization	The candidate finishes the last question and clicks "Finish Interview". The Client calls the API <code>/api/session/finish</code> .	Server (FastAPI)	To finalize the process by updating the <code>meta.json</code> file with the final status and end time.

Chapter 4

SECURITY AND RELIABILITY (UX)

4.1 Security Measures

Security protocols are primarily enforced on the server-side and during data transfer to safeguard the confidentiality and integrity of the video interview data.

4.1.1 Server-Side Validation

To prevent unauthorized access and file uploads, every significant request directed to the backend must be properly authenticated.

- **Mechanism:** All requests to data-handling APIs (such as `/api/upload-one`) are required to include a valid **Token** that the candidate received and used at login.
- **Response:** If the Server receives a request that is missing a Token or contains an invalid Token, the FastAPI system immediately returns a **401 Unauthorized Error**. This essential check ensures that only authenticated users can manipulate session data.

4.1.2 HTTPS/SSL Encryption

Securing the communication channel is a mandatory requirement, particularly when dealing with sensitive multimedia data.

- **Data Protection:** We utilize SSL/TLS encryption (by using Ngrok to create an HTTPS tunnel). This encrypts the video and metadata during transmission from the Client to the Server, effectively preventing unauthorized parties from intercepting the data.
- **Device Access:** Using HTTPS is a **non-negotiable requirement** for modern browsers to grant permission to access the Camera/Microphone via the MediaStream Recording API. Without this secure protocol, the recording function cannot operate remotely.

4.2 Error Handling & Reliability (UX)

The system's dependability is significantly strengthened through active error management, focused on maintaining data integrity during the file upload process.

4.2.1 Manual Retry Policy

This feature is the core defense against data loss caused by temporary network instability, greatly enhancing the User Experience.

- **Error Scenario:** If the network connection is lost or unstable while the video response is being uploaded to the server (Step 3b), the upload will fail.
- **Solution:** The user interface (`interview.html`) detects the failure and automatically displays an error message along with a **"Retry"** button.
- **Efficiency:** The candidate simply clicks this button to resend the exact video data they just recorded, without having to perform the entire recording for that question again. This ensures interview continuity and saves the candidate considerable time and effort.

4.2.2 Input Validation (Client-Side)

Validating user data at the first possible point (the Frontend) helps to reduce unnecessary workload and potential errors on the Backend.

- **Client-Side Check:** On the login page, the JavaScript logic (`login.js`) prevents the user from submitting empty name or token fields.
- **Benefit:** This initial validation ensures that every session started has the required basic identifying information, keeping the Backend processing clean and preventing the creation of empty or confusing storage folders.

Chapter 5

EXPERIMENTAL RESULTS

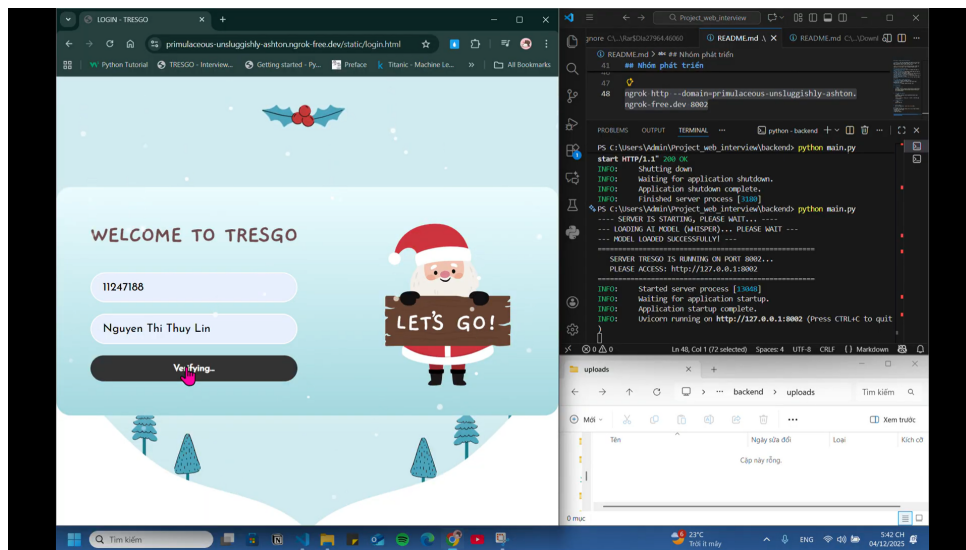


Figure 5.1: Login interface successful.

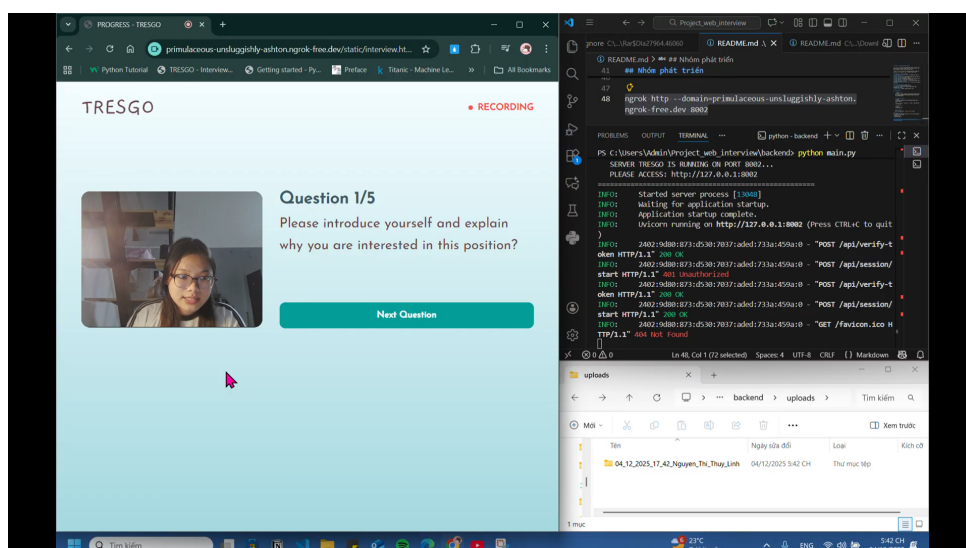


Figure 5.2: Recording interface with status indicator light.

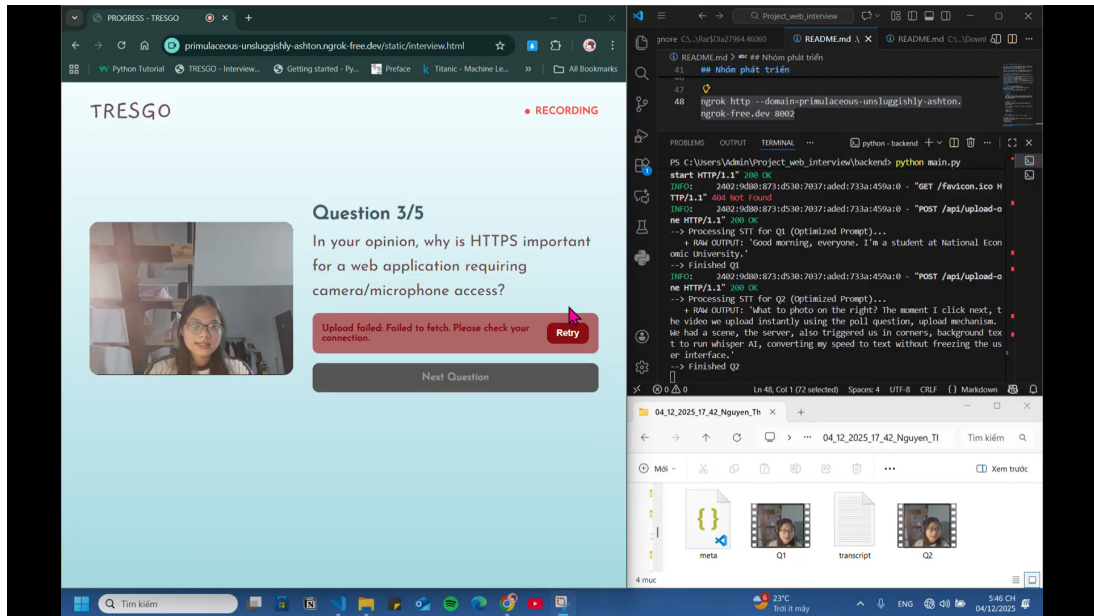


Figure 5.3: Server log displays the process of receiving files and running AI.

Name	Size	Packed	Type	Modified	CRC32
File folder					
transcript.txt	873	465	Text Document	12/4/2025 5:...	1DC982...
Q5.webm	1,627,391	1,627,391	WEBM File	12/4/2025 5:...	9335FFE7
Q4.webm	4,146,980	4,146,980	WEBM File	12/4/2025 5:...	AECB38...
Q3.webm	6,521,508	6,521,508	WEBM File	12/4/2025 5:...	4012DD...
Q2.webm	7,856,058	7,856,058	WEBM File	12/4/2025 5:...	89B9EB71
Q1.webm	3,178,626	3,178,626	WEBM File	12/4/2025 5:...	0A7C5B51
metajson	1,058	334	JSON Source File	12/4/2025 5:...	ED5F1D...

Figure 5.4: The uploads folder contains all the videos and the (transcript.txt file with accurate content..

Chapter 6

SELF-CHECK QUESTIONS

6.1 Why must the Token be validated on the server?

- **Answer:** The token must be validated exclusively on the **Server** because the Client (web browser) is considered an **untrusted environment**.
- **Rationale:** Client-side logic, such as JavaScript code, can easily be inspected and altered by unauthorized users (e.g., hackers). Validating the token on the Server is the only way to guarantee that only **legitimate requests** are allowed to initiate a session or write sensitive video data to the storage system, thereby ensuring backend security and data integrity.

6.2 What are the benefits of per-question recording and upload?

- **Data Preservation:** It ensures that data are saved immediately after each response. If a technical issue (like a power outage or network failure) occurs during Question 5, the data from the previous four questions remains **secure** on the server.
- **Network Efficiency:** Sending multiple small video files sequentially is generally more stable and efficient than attempting to upload one very large file (15–20 minutes), which can easily time

6.3 Why is HTTPS required publicly?

- **Encryption:** HTTPS uses SSL/TLS encryption to protect the transmission of sensitive video and audio data from the Client to the Server against eavesdropping.
- **Device Access:** Modern web browsers strictly block access to the Camera and Microphone via the MediaStream Recording API unless the website is being served over the secure HTTPS protocol. HTTPS is therefore non-negotiable for the core recording feature to function over the public internet.

6.4 How do you design the retry/backoff mechanism?

- **Approach:** When an upload failure occurs due to a network error, the interface clearly displays an error message and a "Retry" button.
- **UX Goal:** This design ensures a clear and transparent User Experience (UX). The candidate can proactively press the button to resend the exact recorded data once they have stabilized their network connection, without being forced to re-record the answer.

6.5 Which Speech-to-Text engine and why?

- **Cost and Privacy:** It is an open-source model that can run offline directly on our server, eliminating dependency on external, costly APIs and maximizing data privacy.
- **Performance:** Whisper offers superior multilingual support, accurately handling both Vietnamese and English audio, which is crucial for interviews.

Chapter 7

CONCLUSION AND FUTURE DEVELOPMENT

7.1 Conclusion

The TresGo video interview system has been successfully developed and deployed using a robust Client-Server architecture. The project effectively addressed the core real-world challenge of data loss by implementing the Per-Question Upload mechanism and rigorous Server-side authentication. The system is stable, meets all initial objectives, and successfully integrates the Whisper AI engine as a non-blocking background task.

7.2 Limitations

The main current limitation of the system lies in the **AI processing speed**. The Speech-to-Text (STT) transcription task, although running in the background, is heavily dependent on the available **CPU resources** of the current server infrastructure. Running the basic Whisper model means that processing multiple concurrent interview sessions could introduce noticeable delays in generating transcripts.

7.3 Future Development

To enhance performance, accuracy, and scalability, the TresGo project should prioritize the following development paths:

- **AI Performance Upgrade:** Migrate the server infrastructure to a platform equipped with a dedicated **GPU**. This upgrade would allow the use of the more powerful **Whisper Large** model, leading to significantly faster processing times and higher transcription accuracy.
- **Scalable Cloud Storage:** Transition the data storage structure from local disk storage to a scalable cloud storage solution, such as **AWS S3** or Google Cloud Storage. This move will boost data durability, increase fault tolerance, and prepare the application for supporting a massive scale of concurrent users.